

1. Introduction to Operating Systems and Kernels

1.1. Overview of Operating Systems An **operating system (OS)** is software that manages a computer's hardware, provides a basis for application programs, and acts as an intermediary between the user and the computer hardware 1, 2. Its fundamental goals are to execute programs in a **convenient and efficient** manner 1. From a system view, the OS is a **resource allocator** that manages conflicting requests for CPU time, memory space, and I/O devices to ensure efficient and fair operation 3, 4.

1.2. Operating System Services The OS provides a variety of services to users and programs, including:

- **User Interface:** Almost all systems have a user interface, ranging from **Graphical User Interfaces (GUI)** and touch-screen interfaces to **Command-Line Interfaces (CLI)** 5-7.
- **Program Execution:** The system must be able to load a program into memory and run it 5.
- **I/O Operations:** Because users cannot control I/O devices directly, the OS provides means to read or write to devices and file systems 5.
- **File-System Manipulation:** This includes creating, deleting, and searching for files and directories 5.
- **Communications:** Facilitates information exchange between processes via **shared memory** or **message passing** 8.
- **Resource Allocation:** When multiple processes run concurrently, the OS manages the allocation of CPU cycles, main memory, and file storage 9.
- **Accounting and Protection:** The OS tracks user activity for logging and ensures that all access to system resources is controlled and secure 6, 9.

1.3. Kernels: Linux Versus Classic Unix Kernels are the innermost portion of the OS, providing basic services for all other parts of the system and managing hardware 10.

- **Monolithic Kernels:** Classic Unix systems are typically monolithic, existing as a single, large executable image that runs entirely in a single address space in kernel mode 11, 12.
- **Microkernels:** These systems move non-essential components from the kernel into user-space "servers" that communicate via message passing 13. While modular, this design can suffer from performance hits due to frequent context switches 14.
- **Linux Evolution:** Linux is a **monolithic kernel** but incorporates many features of microkernels, such as a **modular design**, the ability to preempt itself (**kernel preemption**), and the support for dynamically loading kernel modules 15.
- **Key Differences:** Unlike many traditional Unix variants, Linux supports **symmetrical multiprocessing (SMP)** and treats threads not as distinct from processes, but as tasks that simply happen to share resources 16, 17.

2. Process Concepts and Management

2.1. The Process and Its States A **process** is a program in execution 18, 19. It includes executable code (**text section**), global variables (**data section**), dynamically allocated memory (**heap**), and temporary data storage (**stack**) for function parameters and return addresses 18-20. Processes transition through several states during their life cycle 21, 22:

- **New:** The process is being created 22.

- **Running:** Instructions are being executed 22.
- **Waiting/Blocked:** The process is waiting for an event, such as I/O completion 22, 23.
- **Ready:** The process is waiting to be assigned to a processor 22, 23.
- **Terminated/Zombie:** The process has finished execution but its entry remains in the process table until the parent collects its status 22, 24.

2.2. Process Data Structures Each process is represented by a **Process Control Block (PCB)**, also called a task structure in Linux (struct task_struct) 25, 26. This structure contains all information the kernel needs to manage the process, including its PID, state, program counter, CPU registers, and scheduling information 25-27. The kernel stores these in a circular doubly linked list called the **task list** 25.

2.3. Process API Operating systems provide an API for process management:

- **Creation:** In Unix-like systems, a new process is created via the fork() system call, which creates a copy of the parent process 28, 29. The exec() family of calls is then used to replace the process's memory space with a new program 30, 31.
- **Termination:** A process terminates itself using the exit() system call, which releases its resources to the OS 30, 32.
- **Waiting:** A parent can use wait() to suspend execution until a child process finishes, allowing it to retrieve the child's exit status 30, 33.

2.4. Sharing the Processor: Modes and Context Switching

- **User and Kernel Modes:** To protect the system, hardware provides at least two modes of operation: **user mode** (mode bit 1) for applications and **kernel mode** (mode bit 0) for the OS 34, 35. Privileged instructions can only be executed in kernel mode 36, 37.
- **Context Switching:** When the scheduler switches the CPU from one process to another, it performs a **context switch**, saving the state of the current process in its PCB and restoring the saved state of the next process 38. This is pure overhead as the system does no useful work during the switch 38.

2.5. System Boot Sequence When a computer is powered on, a **bootstrap program** (boot loader) located in firmware (BIOS or UEFI) initializes the system and locates the OS kernel on the **boot disk** 39-41. This program loads the kernel into memory, which then starts the first process (typically systemd or init) and begins providing system services 42-44.

3. Threads and Concurrency

3.1. Concept and Benefits of Threads A **thread** is a basic unit of CPU utilisation that shares the code, data, and resources of its parent process but maintains its own program counter, registers, and stack 45, 46. **Multithreading** offers significant benefits 47-49:

- **Responsiveness:** An application can continue running even if one part is blocked 47.
- **Resource Sharing:** Threads share memory by default, allowing multiple activities within one address space 48.
- **Economy:** Thread creation and context switching are much faster and less resource-intensive than process-level operations 49.
- **Scalability:** Threads can run in parallel on different cores of a multiprocessor system 49.

3.2. Multithreading Models The relationship between user threads and kernel threads is managed through three models 50, 51:

- **Many-to-One:** Many user threads map to a single kernel thread; if one thread blocks, the entire process blocks 51, 52.
- **One-to-One:** Each user thread maps to a kernel thread, providing the best concurrency 53. This is used by **Linux and Windows** 53.
- **Many-to-Many:** Multiplexes many user threads to a smaller or equal number of kernel threads 54.

3.3. Case Study: Linux Implementation of Threads Linux implements threads in a unique way: it does not distinguish between threads and processes 55, 56. To the kernel, both are **tasks** 55, 56. Threads are created using the clone() system call with specific flags (like CLONE_VM, CLONE_FS) that allow the new task to share the parent's address space and file-system information 57-59.

4. Process Scheduling

4.1. Scheduling Concepts and Algorithms The **CPU scheduler** selects a process from the **ready queue** to execute on an available core 60, 61. Scheduling can be **non-preemptive** (the process holds the CPU until it terminates or blocks) or **preemptive** (the OS can forcibly remove the process from the CPU) 62, 63. Common algorithms include:

- **First-Come, First-Served (FCFS):** Simple but can lead to long waiting times if a long process blocks others (the **convoy effect**) 64, 65.
- **Shortest-Job-First (SJF):** Assigns the CPU to the process with the smallest next burst; it is provably optimal for minimizing average waiting time 66, 67.
- **Round-Robin (RR):** Each process gets a small unit of time (**time quantum**); if it isn't finished, it is preempted and put back in the ready queue 68, 69.

4.2. Multi-Level Feedback Queue (MLFQ) The MLFQ is a complex scheduling algorithm that uses multiple queues with different priority levels 70. A process can move between queues based on its behavior: if it uses too much CPU time, it is moved to a lower-priority queue; if it waits too long at a low level, it is "aged" and moved up to prevent **starvation** 70-72.

4.3. Case Study: Linux Completely Fair Scheduler (CFS) The **CFS** is the default Linux scheduler for normal tasks 73, 74. Instead of traditional time slices, it assigns each thread a **proportion of the processor's time** based on its **nice value** 75, 76.

- **Virtual Runtime (vruntime):** The CFS tracks the vruntime of each task, representing the actual runtime weighted by its priority 77, 78.
- **Selection:** The scheduler always picks the task with the **smallest vruntime** to run next 78, 79.
- **Implementation:** To find the task with the lowest vruntime efficiently, Linux uses a **red-black tree** (a self-balancing binary search tree) 79, 80.

Analogy: You can think of the **CPU scheduler** as a teacher managing a classroom where many students (processes) want to ask questions 60, 81. The **First-Come, First-Served** approach is a simple line, but if one student has a very long question, everyone else waits a long time 64, 65. **Round-Robin** is like giving every student 30 seconds to speak before they have to go to the back of the line, ensuring everyone is heard eventually 68, 82.